

Week 1 - Day 5c: Hangman

This is the final exercise for this week. We will combine many of the basic programming concepts we have learned so far to implement two famous games. Some of the concepts that we are going to be using are:

- **Variables**
- **Data types**
- **Conditionals**
- **Loops**
- **Functions**

☐ Only start working on this exercise sheet if you have completed all material (including the supplementaries) until and including day 4.

Game: Hangman [] [] [] []

We will try to implement the famous [Hangman](#) game, so that you can play the game against the computer. As a reminder, the computer will choose a word and you will have to guess it by suggesting letters within a certain number of attempts. We will write a simple game loop that steps through the logic of the game. To make this task a bit more manageable we will have you write helper functions that implement some part of the logic which we can check separately.

This type of software design is called bottom-up design. You start by writing the small parts of the programs which you test independently and then you combine them to create the final program.

Specifically, we will implement the following functions:

- `get_word`: This function will return a random word from a predefined list of words.
- `print_state`: This function will display the game state to the user, showing the letters that have been guessed and replacing the others with a placeholder (e.g. `_`). It also shows the number of attempts left.
- `is_game_ended`: This will be a helper function that checks if the game has ended. The game ends when the player has guessed the word or when the number of attempts reaches zero.
- `get_guess`: This function will get the user's guess and check if it is a valid guess.
- `update_state`: This function will update the game state based on the user's guess.

Finally, we will combine these functions in a game loop that will run until the game ends.

1.1 We start with the `get_random_word` function which chooses a random word from a list of words. Because we haven't dealt with files yet, we have implemented this function for you.

```
import random

def get_random_word():
```

```

"""Selects a random word from a list in uppercase."""
# Read the list of words from a file
with open("assets/nouns.txt", "r") as f:
    words = f.read().split()

# Choose a random word from the list
chosen_word = random.choice(words).upper()

return chosen_word

# Test the function here

```

1.2 Next, we want to be able to display the state of the game to the user. Write a function `display_state` that takes the following arguments:

- `word`: a string representing the word to guess
- `guessed`: a list of strings representing the letters that have been guessed so far
- `attempts`: an integer representing the number of attempts left

Given these arguments it should print out the current state of the game. For example, if the word is HELLO, the guessed letters are ['H', 'E', 'L'] and the number of attempts left is 2, the function should print:

```
H E L L _ (2 attempts left)
```

Note, you do not have to worry about letter casing. You can assume (and will later implement) that the user input will always be uppercase alphabetic characters.

```

# TODO: Your code here

# Test your function here

```

1.3 Game loops are typically implemented using a `while` loop. The game should continue until the player has guessed the word or has run out of attempts. It is important that these loops have clear exit conditions. For this game the exit condition is when the player has guessed the word or has run out of attempts. Let's implement both of these conditions in two functions:

- `no_attempts_left`: This function should return `True` if there are no attempts left. It only takes the integer `attempts` as an argument which represents the number of attempts left.
- `word_guessed`: This function should return `True` if the word has been guessed. The function takes the string `word` (the word to guess) and `guessed`, a list of strings representing the letters that have been guessed so far.

```

# TODO: Your code here (no_attempts_left)

# TODO: Your code here (word_guessed)

```

Finally, we can combine these boolean conditions to check if the game has ended. Implement a function `is_game_over` which combines the two functions above and returns `True` if the game has ended.

```
# TODO: Your code here (is_game_over)

# Test your function here
```

1.4 A main aspect of the game is to get the user to guess letters. However, this is also a potential crux. Users might (unintentionally) enter non-alphabetic letters, like numbers, special characters, or even multiple letters. If these are not handled correctly, the game might crash or behave unexpectedly. To prevent this, we need to validate the user input.

We want to implement a function `get_guess` that asks the user for a single letter input. If the user enters a valid lower- or uppercase alphabetic letter, the function should return the uppercase version of the letter. If the user enters anything else, the function should print a descriptive error message and return `None`. Please test for the following exceptions:

1. Single Letters: If the user enters more than one letter or an empty string, the function should print "Please enter exactly one letter."
2. Non-Alphabetic Characters: If the user enters a non-alphabetic character, the function should print "Please enter a letter."
3. Empty Input: If the user enters nothing, the function should print "Please enter a letter."
4. Already Guessed: If the user enters a letter that has already been guessed, the function should print "You have already guessed this letter."

For the last check, you need the guessed letters as an additional argument. You can assume that the guessed letters are stored in a list of strings.

```
# TODO: Your code here

# Test your function here
```

1.5 Finally, we can piece together the main game. First, you have to set the initial state, i.e. the word to guess, the guessed letters, and the number of attempts. Then, you can start the game loop. In each iteration of the loop, you print the current state, get a guess from the user, and update the state. If the game is ended, you print the result.

```
# TODO: Fill out the remaining logic
from IPython.display import clear_output

def hangman():
    # Initialize the game
    word = get_random_word()
    guessed = []
    attempts = 5 # Feel free to change this to adjust difficulty

    while True:
        # Clear output
```

```
clear_output()
```

```
# TODO: Your code here
```

```
hangman()
```

```
Y O U R (2 attempts left)
```

```
Congratulations! You guessed the word: YOUR
```